

ASSIGNMENT 6.5

G.ARNAV

2303A51928

BATCH-30

#TASK-1

TASK:

Use an AI tool to generate eligibility logic.

PROMPT:

Generate Python code to check voting eligibility based on age and citizenship

CODE:

```
#Generate Python code to check voting eligibility based on age and citizenship status
def check_voting_eligibility(age, is_citizen):
    if age >= 18 and is_citizen:
        return "You are eligible to vote."
    elif age < 18:
        return "You are not eligible to vote due to age."
    else:
        return "You are not eligible to vote due to citizenship status."
# Example usage
age = int(input("Enter your age: "))
citizenship_input = input("Are you a citizen? (yes/no): ").strip().lower()
is_citizen = citizenship_input == 'yes'
print(check_voting_eligibility(age, is_citizen))
```

OUTPUT:

```
PS C:\200126> & C:/Users/gunda/AppData/Local/Programs/Python/Python313/python.exe c:/200126/assignment6_5.py
Enter your age: 17
Are you a citizen? (yes/no): yes
You are not eligible to vote due to age.
```

OBSERVATION:

The program successfully accepts user input for age and citizenship status and processes it correctly. By converting the input to lowercase and removing extra spaces, the code avoids errors caused by different user input formats. This ensures smooth execution and accurate decision-making without crashing the program.

#TASK-2

TASK:

Use an AI tool to process strings using loops.

PROMPT:

Generate Python code to count vowels and consonants in a string using a loop

CODE:

```
#Generate Python code to count vowels and consonants in a string using a loop
def count_vowels_consonants(input_string):
    vowels = 'aeiouAEIOU'
    vowel_count = 0
    consonant_count = 0

    for char in input_string:
        if char.isalpha(): # Check if the character is a letter
            if char in vowels:
                vowel_count += 1
            else:
                consonant_count += 1

    return vowel_count, consonant_count
# Example usage
input_string = input("Enter a string: ")
vowels, consonants = count_vowels_consonants(input_string)
print(f"Vowels: {vowels}, Consonants: {consonants}")
```

OUTPUT:

```
PS C:\200126> & C:/Users/gunda/AppData/Local/Programs/Python/Python313/python.exe c:/200126/assignment6_5.py
Enter a string: arnav
Vowels: 2, Consonants: 3
PS C:\200126> █
```

OBSERVATION:

The program loops through each character in the given string and checks whether it is a vowel or a consonant. It counts and displays the total number of vowels and consonants, showing how loops and conditional statements work together to process text efficiently.

#TASK-3

TASK:

Use an AI tool to generate a complete program using classes, loops, and conditionals.

PROMPT:

Generate a Python program for a library management system using classes, loops, and conditional statements

CODE:

```
#Generate a Python program for a library management system using classes, loops, and conditional statements
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author
        self.is_available = True

class Library:
    def __init__(self):
        self.books = []

    def add_book(self, book):
        self.books.append(book)

    def borrow_book(self, title):
        for book in self.books:
            if book.title == title:
                if book.is_available:
                    book.is_available = False
                    return f"You have borrowed '{book.title}' by {book.author}."
                else:
                    return f"'{book.title}' is currently unavailable."
        return "Book not found."

    def return_book(self, title):
        for book in self.books:
            if book.title == title:
                if not book.is_available:
                    book.is_available = True
                    return f"You have returned '{book.title}'."
                else:
                    return f"'{book.title}' was not borrowed."
        return "Book not found."

# Example usage
library = Library()
library.add_book(Book("The Great Gatsby", "F. Scott Fitzgerald"))
library.add_book(Book("To Kill a Mockingbird", "Harper Lee"))
print(library.borrow_book("The Great Gatsby"))
```

OUTPUT:

```
PS C:\200126> & C:/Users/gunda/AppData/Local/Programs/Python/Python313/python.exe c:/200126/assignment6_5.py
You have borrowed 'The Great Gatsby' by F. Scott Fitzgerald.
PS C:\200126> █
```

OBSERVATION:

The program uses classes to represent books and library operations, while loops and conditional statements help manage borrowing and returning books. This makes the library system organized, easy to understand, and efficient for basic management tasks.

#TASK-4

TASK:

Use an AI tool to generate an attendance management class.

PROMPT:

Generate a Python class to mark and display student attendance using loop.

CODE:

```
#Generate a Python class to mark and display student attendance using loop
class Student:
    def __init__(self, name):
        self.name = name
        self.attendance = []

    def mark_attendance(self, date):
        self.attendance.append(date)

    def display_attendance(self):
        print(f"Attendance for {self.name}:")
        for date in self.attendance:
            print(f"- {date}")

# Example usage
student = Student("Alice")
student.mark_attendance("2024-01-01")
student.mark_attendance("2024-01-02")
student.display_attendance()
```

OUTPUT:

```
You have borrowed "The Great Gatsby" by F. Scott Fitzgerald.
PS C:\200126> & C:/Users/gunda/AppData/Local/Programs/Python/Python313/python.exe c:/200126/assignment6_5.py
Attendance for Alice:
- 2024-01-01
- 2024-01-02
```

OBSERVATION:

The program uses a class to store student details and a loop to mark attendance for each student. It then displays the attendance clearly, showing how classes and loops work together to manage student records efficiently.

#TASK-5

TASK:

Use an AI tool to complete a navigation menu

PROMPT:

Generate a Python program using loops and conditionals to simulate an ATM menu

CODE:

```
#Generate a Python program using loops and conditionalsto simulate an ATM menu and generate direct output
balance = 5000

while True:
    print("\n-- ATM Menu ---")
    print("1. Check Balance")
    print("2. Deposit Money")
    print("3. Withdraw Money")
    print("4. Exit")

    choice = input("Enter your choice (1-4): ")

    if choice == "1":
        print("Your balance is:", balance)

    elif choice == "2":
        amount = int(input("Enter amount to deposit: "))
        if amount > 0:
            balance += amount
            print("Amount deposited successfully.")
        else:
            print("Invalid deposit amount.")

    elif choice == "3":
        amount = int(input("Enter amount to withdraw: "))
        if amount <= balance and amount > 0:
            balance -= amount
            print("Please collect your cash.")
        else:
            print("Insufficient balance or invalid amount.")

    elif choice == "4":
        print("Thank you for using the ATM.")
        break

    else:
        print("Invalid choice. Please try again.")
```

OUTPUT:

```
PS C:\200126> & C:/Users/gunda/AppData/Local/Programs/Python/Python313/python.exe c:/200126/assignment6_5.py

--- ATM Menu ---
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Enter your choice (1-4):
```

OBSERVATION:

The program uses loops to repeatedly display the ATM menu and conditional statements to perform actions based on the user's choice. This allows smooth interaction and ensures the correct operation is selected until the user decides to exit.

